

Cours Développement Web

Niveau Intermédiaire • Semaine 4 – Activité n°4

Données : MySQL, SQL avancé, administration et PDO/MySQLi

Durée : 06 heures | Modalité : individuel | Remise : ZIP

Objectifs	Expérimenter GET/POST, créer et administrer une base MySQL, écrire des requêtes SQL (SELECT/INSERT/UPDATE/DELETE), puis connecter PHP à MySQL via PDO et MySQLi en appliquant de bonnes pratiques de sécurité (requêtes préparées).
Contenu visé	Module 4.1 à 4.6 : GET/POST, MySQL & SQL, phpMyAdmin, PDO, MySQLi, requêtes préparées, prévention injection SQL.
Ressources	Ressources du module + Laragon + phpMyAdmin + documentation PHP/MySQL (si besoin).
Type de l'activité	Activité individuelle.
Outils	Laragon (PHP 8.x + MySQL), phpMyAdmin, VS Code, navigateur, DevTools.
Résultat attendu	Un fichier .zip contenant : (A) mini-atelier guidé (GET/POST + création DB); (B) activité principale : base de données + scripts PHP PDO & MySQLi + README.
Durée / Évaluation	Durée recommandée : 06 heures (3h atelier + 3h activité). Évaluation par le tuteur : conformité, fonctionnement, sécurité, clarté du code et organisation.

1. Présentation

Cette activité est structurée en deux volets :

- Volet A (atelier avant) : expérimenter GET/POST et créer une base MySQL via phpMyAdmin (avec solutions au fur et à mesure).
- Volet B (activité principale) : Concevoir et créer une base de données complète et coder la connexion + PDO avec MySQLi (requêtes préparées et bonnes pratiques).

2. Volet A — Atelier guidé (avec solutions) : GET/POST + phpMyAdmin

Objectif : démarrer rapidement sur un environnement local (Laragon) et comprendre le cycle : formulaire → requête HTTP → récupération PHP → affichage sécurisé ; puis créer une base simple dans phpMyAdmin.

2.1 Préparation (Laragon + phpMyAdmin)

1. Lancer Laragon puis cliquer sur Start (serveur + MySQL).
2. Vérifier : `http://localhost/` s'ouvre dans le navigateur.
3. Ouvrir phpMyAdmin : Menu Laragon → Database → phpMyAdmin (ou `http://localhost/phpmyadmin`).
4. Créer un dossier de travail : `C:\laragon\www\atelier_donnees/`

2.2 Exercice A1 — GET (formulaire + page résultat)

- Créer `get_form.php` : formulaire `method=GET`.
- Créer `get_result.php` : récupérer `$_GET` et afficher les valeurs.
- BON réflexe : afficher avec `htmlspecialchars()` (prévention XSS).

Explication de l'énoncé : Pourquoi `htmlspecialchars()` ? – Prévention des failles XSS

Lorsque vous affichez des données saisies par un utilisateur (dans notre cas via `$_GET`), vous ne devez jamais faire confiance à leur contenu. Un utilisateur malveillant pourrait envoyer, par exemple, un champ contenant du code JavaScript comme :

```
<script>alert('Hack !');</script>
```

Si vous affichez cette valeur directement dans la page HTML (avec `echo` ou `print`), le navigateur interprétera ce code et l'exécutera. C'est ce qu'on appelle une **faille XSS (Cross-Site Scripting)**. Elle peut permettre le vol de session, la redirection vers des sites frauduleux, etc.

Le réflexe à adopter : `htmlspecialchars()`

La fonction `htmlspecialchars()` convertit les caractères spéciaux du HTML en **entités HTML**. Par exemple :

- `<` devient `<`;
- `>` devient `>`;
- `"` devient `"`;
- `'` devient `'` (ou `'`);
- `&` devient `&`;

Ainsi, le code `<script>` devient `<script>` : il s’affiche à l’écran comme un texte inoffensif au lieu d’être exécuté.

Solution A1 — get_form.php

```
<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>GET</title></head>
<body>
  <h1>Exemple GET</h1>
  <form action="get_result.php" method="get">
    <label>Nom : <input type="text" name="nom"></label><br>
    <label>Ville : <input type="text" name="ville"></label><br>
    <button type="submit">Envoyer</button>
  </form>
</body>
</html>
```

Solution A1 — get_result.php

```
<?php
$nom = trim($_GET['nom'] ?? '');
$ville = trim($_GET['ville'] ?? '');

echo "<h1>Résultat GET</h1>";
echo "Nom : " . htmlspecialchars($nom, ENT_QUOTES, 'UTF-8') . "<br>";
echo "Ville : " . htmlspecialchars($ville, ENT_QUOTES, 'UTF-8') . "<br>";
```

2.3 Exercice A2 — POST (formulaire + validation simple)

- Créer `post_form.php` : formulaire `method=POST`.
- Créer `post_result.php` : récupérer `$_POST` et valider (email).
- Afficher un message d’erreur si la validation échoue.

Solution A2 — post_form.php

```
<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>POST</title></head>
<body>
  <h1>Exemple POST</h1>
  <form action="post_result.php" method="post">
    <label>Email : <input type="text" name="email"></label><br>
    <label>Message : <textarea name="message"></textarea></label><br>
    <button type="submit">Envoyer</button>
  </form>
</body>
</html>
```

Solution A2 — post_result.php

```
<?php
$email = trim($_POST['email'] ?? '');
$message = trim($_POST['message'] ?? '');

echo "<h1>Résultat POST</h1>";

if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Erreur : email invalide.";
    exit;
}

echo "Email : " . htmlspecialchars($email, ENT_QUOTES, 'UTF-8') . "<br>";
echo "Message : " . nl2br(htmlspecialchars($message, ENT_QUOTES, 'UTF-8'));
```

2.4 Exercice A3 – Formulaire complet + validation + GET/POST + affichage stylé

1. Contexte & principe

Dans une application web, un formulaire permet de collecter des données. Ces données sont envoyées au serveur via une requête HTTP : • GET (données dans l'URL) ou • POST (données dans le corps de la requête). Dans cet atelier, vous réalisez le même formulaire deux fois : une version GET et une version POST. Vous validez les données côté serveur (PHP) et affichez : soit les erreurs, soit les données saisies dans une page mise en forme en CSS.

2. Mise en place (Laragon) — où mettre le code et comment tester

1. Démarrer Laragon → Start.
2. Créer un dossier : C:\laragon\www\atelier_get_post_form\
3. Ouvrir ce dossier dans l'éditeur.
4. Dans le navigateur, tester l'accès : http://localhost/atelier_get_post_form/

Arborescence proposée

```
atelier_get_post_form/
  index.php
  form_get.php
  form_post.php
  handle_get.php
  handle_post.php
  css/
```

style.css

3. Étape 1 — Créer le style CSS (solution)

- Créer css/style.css.
- Ce style sera utilisé par les pages formulaire et résultat.

Solution — css/style.css

```
/* css/style.css */
:root{
  --bg:#0b1220; --panel:#0f1a2f; --text:#e9eefc; --muted:#b6c2e2;
  --ok:#1b5e20; --err:#b00020; --border:rgba(255,255,255,.12);
  --accent:#4cc3ff; --radius:12px;
}
*{box-sizing:border-box}
body{margin:0;font-family:Arial,sans-serif;background:var(--bg);color:var(--text)}
.container{max-width:860px;margin:24px auto;padding:0 16px}
.card{background:var(--panel);border:1px solid var(--border);border-radius:var(--radius);padding:16px;margin:16px 0}
.hint{color:var(--muted);font-size:14px}
label{display:block;margin:10px 0 6px 0}
input,select,textarea{width:100%;max-width:520px;padding:10px;border-radius:10px;border:1px solid var(--border);background:#0b162a;color:var(--text)}
textarea{min-height:90px}
.row{display:flex;gap:16px;flex-wrap:wrap}
.row > div{flex:1;min-width:240px}
.btn{display:inline-block;padding:10px 14px;border:0;border-radius:10px;background:var(--accent);color:#031019;font-weight:700;cursor:pointer}
.btn.secondary{background:transparent;color:var(--text);border:1px solid var(--border)}
.alert{padding:10px 12px;border-radius:10px;margin:10px 0}
.alert.error{background:rgba(176,0,32,.15);border:1px solid rgba(176,0,32,.35)}
.alert.ok{background:rgba(27,94,32,.18);border:1px solid rgba(27,94,32,.35)}
.kv{display:grid;grid-template-columns:180px 1fr;gap:10px;margin-top:10px}
.kv div{padding:8px;border-bottom:1px dashed rgba(255,255,255,.10)}
small{color:var(--muted)}
a{color:var(--accent);text-decoration:none}
a:hover{text-decoration:underline}
```

4. Étape 2 — Page d'accueil (solution)

- Créer index.php : propose deux liens vers les versions GET et POST.

Solution — index.php

```
<?php // index.php ?>
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Atelier GET/POST</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <div class="container">
    <div class="card">
      <h1>Atelier — Formulaire GET/POST</h1>
      <p class="hint">Choisissez une version :</p>
      <p>
        <a class="btn" href="form_get.php">Version GET</a>
        <a class="btn secondary" href="form_post.php">Version POST</a>
      </p>
    </div>
  </div>
</body>
</html>
```

Ce fichier index.php sert de menu principal. il faut créer maintenant form_get.php et form_post.php avec les formulaires correspondants, ainsi que les pages résultat (get_result.php, post_result.php). Ainsi nous permet de structurer l'atelier : une page d'accueil qui redirige vers les deux exercices distincts (GET / POST).

5. Étape 3 — Créer le formulaire (types de champs) : consigne

- Créer un formulaire contenant :
 - - zone de saisie (nom complet),
 - - email,
 - - nombre (âge),
 - - combo box <select> (niveau),

- - liste modifiable <datalist> (ville),
- - checkbox (conditions),
- - textarea (message).

Règles de validation (à appliquer côté serveur)

- Nom complet : obligatoire, 5 à 30 caractères.
- Email : obligatoire, format valide.
- Âge : obligatoire, entre 10 et 99.
- Niveau : obligatoire, valeurs autorisées : débutant / intermédiaire / avancé.
- Ville : obligatoire, 2 à 40 caractères.
- Conditions : obligatoire (case cochée).
- Message : optionnel, si présent alors ≥ 10 caractères.

6. Étape 4 — Version GET : formulaire + traitement (solutions)

6.1 Solution — form_get.php

```
<?php // form_get.php ?>
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Formulaire GET</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
  <div class="card">
    <h1>Formulaire (GET)</h1>
    <p class="hint">Les données seront visibles dans l'URL (query string).</p>

    <form action="handle_get.php" method="get">
      <div class="row">
        <div>
          <label for="nom">Nom complet</label>
          <input id="nom" name="nom" type="text" placeholder="Ex : Amine Ben Ali">
```

```

</div>
<div>
  <label for="email">Email</label>
  <input id="email" name="email" type="text" placeholder="Ex : amine@mail.com">
</div>
</div>

<div class="row">
  <div>
    <label for="age">Âge</label>
    <input id="age" name="age" type="number" min="10" max="99" placeholder="Ex : 22">
  </div>
  <div>
    <label for="niveau">Niveau</label>
    <select id="niveau" name="niveau">
      <option value="">-- Choisir --</option>
      <option value="debutant">Débutant</option>
      <option value="intermediaire">Intermédiaire</option>
      <option value="avance">Avancé</option>
    </select>
  </div>
</div>

<label for="ville">Ville (liste modifiable)</label>
<input id="ville" name="ville" list="villes" placeholder="Commencez à taper...">
<datalist id="villes">
  <option value="Tunis"></option>
  <option value="Sfax"></option>
  <option value="Sousse"></option>
  <option value="Bizerte"></option>
</datalist>

<label for="message">Message (optionnel)</label>
<textarea id="message" name="message" placeholder="Votre message..."></textarea>

<label>
  <input type="checkbox" name="cgu" value="1"> J'accepte les conditions
</label>

<button class="btn" type="submit">Envoyer (GET)</button>
<a class="btn secondary" href="index.php">Retour</a>
</form>

```



```
</div>
</div>
</body>
</html>
```

6.2 Solution — handle_get.php (validation + affichage)

```
<?php
// handle_get.php

function e($v){ return htmlspecialchars((string)$v, ENT_QUOTES, 'UTF-8'); }

$nom = trim($_GET['nom'] ?? '');
$email = trim($_GET['email'] ?? '');
$age = trim($_GET['age'] ?? '');
$niveau = trim($_GET['niveau'] ?? '');
$ville = trim($_GET['ville'] ?? '');
$message = trim($_GET['message'] ?? '');
$cgu = isset($_GET['cgu']);

$erreurs = [];

if ($nom === '' || mb_strlen($nom) < 5 || mb_strlen($nom) > 30) {
    $erreurs['nom'] = 'Nom invalide (5 à 30 caractères).';
}
if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $erreurs['email'] = 'Email invalide.';
}
if ($age === '' || !ctype_digit($age) || (int)$age < 10 || (int)$age > 99) {
    $erreurs['age'] = 'Âge invalide (10 à 99).';
}
$nivOK = ['debutant', 'intermediaire', 'avance'];
if ($niveau === '' || !in_array($niveau, $nivOK, true)) {
    $erreurs['niveau'] = 'Niveau invalide.';
}
if ($ville === '' || mb_strlen($ville) < 2 || mb_strlen($ville) > 40) {
    $erreurs['ville'] = 'Ville invalide (2 à 40 caractères).';
}
if (!$cgu) {
    $erreurs['cgu'] = 'Vous devez accepter les conditions.';
}
```

```

if ($message != " " && mb_strlen($message) < 10) {
    $erreurs['message'] = 'Message trop court (min 10 caractères).';
}

?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Résultat GET</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
    <div class="card">
        <h1>Résultat (GET)</h1>

        <?php if (!empty($erreurs)): ?>
            <div class="alert error">
                <strong>Erreurs détectées :</strong>
                <ul>
                    <?php foreach ($erreurs as $err): ?>
                        <li><?= e($err) ?></li>
                    <?php endforeach; ?>
                </ul>
            </div>
            <p><a class="btn" href="form_get.php">Retour au formulaire</a></p>

        <?php else: ?>
            <div class="alert ok">Données valides </div>

            <div class="kv">
                <div><strong>Nom</strong></div><div><?= e($nom) ?></div>
                <div><strong>Email</strong></div><div><?= e($email) ?></div>
                <div><strong>Âge</strong></div><div><?= e($age) ?></div>
                <div><strong>Niveau</strong></div><div><?= e($niveau) ?></div>
                <div><strong>Ville</strong></div><div><?= e($ville) ?></div>
                <div><strong>Message</strong></div><div><?= e($message) ?></div>
            </div>

```

```
<p class="hint"><small>Remarque : en GET, les données sont visibles dans
l'URL.</small></p>
```

```
<p>
  <a class="btn" href="form_get.php">Nouveau test</a>
  <a class="btn secondary" href="index.php">Accueil</a>
</p>
<?php endif; ?>
</div>
</div>
</body>
</html>
```

Ce fichier récupère les données du formulaire GET (\$_GET['nom'], \$_GET['email'], etc.) grâce à l'opérateur ?? qui évite les erreurs si un champ est absent. Une fonction utilitaire e() est définie : elle convertit chaque affichage avec htmlspecialchars() pour bloquer les failles XSS. Ensuite, une série de contrôles (longueur, format email, âge numérique, valeur parmi une liste, case à cocher obligatoire) remplit un tableau \$erreurs. Si des erreurs existent, la page les affiche sous forme de liste ; sinon, elle montre les valeurs saisies dans un tableau « clé/valeur ». L'intérêt pédagogique est de montrer **à la fois** la récupération des paramètres GET, la validation côté serveur, et le réflexe systématique d'échappement à l'affichage.

7. Étape 5 — Version POST : formulaire + traitement (solutions)

7.1 Solution — form_post.php

```
<?php // form_post.php ?>
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Formulaire POST</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
  <div class="card">
    <h1>Formulaire (POST)</h1>
    <p class="hint">Les données ne sont pas visibles dans l'URL.</p>
```

```
<form action="handle_post.php" method="post">
<div class="row">
  <div>
    <label for="nom">Nom complet</label>
    <input id="nom" name="nom" type="text" placeholder="Ex : Amine Ben Ali">
  </div>
  <div>
    <label for="email">Email</label>
    <input id="email" name="email" type="text" placeholder="Ex : amine@mail.com">
  </div>
</div>

<div class="row">
  <div>
    <label for="age">Âge</label>
    <input id="age" name="age" type="number" min="10" max="99" placeholder="Ex : 22">
  </div>
  <div>
    <label for="niveau">Niveau</label>
    <select id="niveau" name="niveau">
      <option value="">-- Choisir --</option>
      <option value="debutant">Débutant</option>
      <option value="intermediaire">Intermédiaire</option>
      <option value="avance">Avancé</option>
    </select>
  </div>
</div>

<label for="ville">Ville (liste modifiable)</label>
<input id="ville" name="ville" list="villes" placeholder="Commencez à taper...">
<datalist id="villes">
  <option value="Tunis"></option>
  <option value="Sfax"></option>
  <option value="Sousse"></option>
  <option value="Bizerte"></option>
</datalist>

<label for="message">Message (optionnel)</label>
<textarea id="message" name="message" placeholder="Votre message..."></textarea>

<label>
  <input type="checkbox" name="cgu" value="1"> J'accepte les conditions
```

```

</label>

<button class="btn" type="submit">Envoyer (POST)</button>
<a class="btn secondary" href="index.php">Retour</a>
</form>
</div>
</div>
</body>
</html>

```

Ce fichier contient un formulaire HTML dont la méthode est POST (method="post"). Contrairement à GET, les données saisies ne seront **pas visibles dans l'URL** (elles voyagent dans le corps de la requête HTTP). Le formulaire envoie les informations vers handle_post.php (traitement côté serveur). On y retrouve différents types de champs : texte (nom, email, ville), numérique (age), liste déroulante (niveau), zone de texte (message) et case à cocher (cgu). Une balise <datalist> associée au champ ville permet une saisie semi-automatique avec suggestions (Tunis, Sfax...). L'intérêt pédagogique est de montrer la **construction d'un formulaire POST** et de faire le lien avec la page de traitement qui récupérera les données via \$_POST.

7.2 Solution — handle_post.php (mêmes validations)

```

<?php
// handle_post.php

function e($v){ return htmlspecialchars((string)$v, ENT_QUOTES, 'UTF-8'); }

$nom = trim($_POST['nom'] ?? '');
$email = trim($_POST['email'] ?? '');
$age = trim($_POST['age'] ?? '');
$niveau = trim($_POST['niveau'] ?? '');
$ville = trim($_POST['ville'] ?? '');
$message = trim($_POST['message'] ?? '');
$cgu = isset($_POST['cgu']);

$erreurs = [];

if ($nom === '' || mb_strlen($nom) < 5 || mb_strlen($nom) > 30) {
    $erreurs['nom'] = 'Nom invalide (5 à 30 caractères).';
}
if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $erreurs['email'] = 'Email invalide.';
}

```

```

if ($age === '' || !ctype_digit($age) || (int)$age < 10 || (int)$age > 99) {
    $erreurs['age'] = 'Âge invalide (10 à 99).';
}
$nivOK = ['debutant','intermediaire','avance'];
if ($niveau === '' || !in_array($niveau, $nivOK, true)) {
    $erreurs['niveau'] = 'Niveau invalide.';
}
if ($ville === '' || mb_strlen($ville) < 2 || mb_strlen($ville) > 40) {
    $erreurs['ville'] = 'Ville invalide (2 à 40 caractères).';
}
if (!$cgu) {
    $erreurs['cgu'] = 'Vous devez accepter les conditions.';
}
if ($message !== '' && mb_strlen($message) < 10) {
    $erreurs['message'] = 'Message trop court (min 10 caractères).';
}

?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Résultat POST</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
    <div class="card">
        <h1>Résultat (POST)</h1>

        <?php if (!empty($erreurs)): ?>
        <div class="alert error">
            <strong>Erreurs détectées :</strong>
            <ul>
                <?php foreach ($erreurs as $err): ?>
                    <li><?= e($err) ?></li>
                <?php endforeach; ?>
            </ul>
        </div>
        <p><a class="btn" href="form_post.php">Retour au formulaire</a></p>

```

```

<?php else: ?>
    <div class="alert ok">Données valides </div>

    <div class="kv">
        <div><strong>Nom</strong></div><div><?= e($nom) ?></div>
        <div><strong>Email</strong></div><div><?= e($email) ?></div>
        <div><strong>Âge</strong></div><div><?= e($age) ?></div>
        <div><strong>Niveau</strong></div><div><?= e($niveau) ?></div>
        <div><strong>Ville</strong></div><div><?= e($ville) ?></div>
        <div><strong>Message</strong></div><div><?= e($message) ?></div>
    </div>

    <p class="hint"><small>Remarque : en POST, les données ne sont pas visibles dans
l'URL.</small></p>
    <p>
        <a class="btn" href="form_post.php">Nouveau test</a>
        <a class="btn secondary" href="index.php">Accueil</a>
    </p>
    <?php endif; ?>
</div>
</div>
</body>
</html>

```

Ce fichier est le traitement du formulaire POST. Il récupère les données via `$_POST` (contrairement à `$_GET`), ce qui les rend invisibles dans l'URL – plus adapté pour les mots de passe ou les messages longs. On retrouve la même fonction `e()` pour protéger l'affichage contre les failles XSS, et les mêmes règles de validation (longueur, format email, âge numérique, liste de niveaux, case CGU obligatoire). En cas d'erreur, un message listant les problèmes est affiché ; sinon, les valeurs saisies s'affichent dans un tableau. L'intérêt pédagogique est de comparer GET vs POST : le code de traitement est presque identique, seule la superglobale change (`$_POST` au lieu de `$_GET`), mais le niveau de sécurité et la discrétion des données diffèrent.

8. Étape 6 — Travail demandé (à faire par l'apprenant)

- Tester la version GET puis la version POST avec plusieurs cas : valeurs valides et valeurs invalides.
- Ajouter au moins 2 validations supplémentaires (au choix) : ex. téléphone, code postal, sélection multiple...

- Améliorer la mise en forme CSS (cartes, couleurs, titres) sans alourdir la page.
- Ajouter une page 'resume.php' qui affiche un résumé du formulaire (facultatif).

2.5 Exercice A3 — Création d'une base simple via phpMyAdmin

Nous proposons une base très simple 'gestion_contacts' (utile pour CRUD) : une table contacts et une table messages. Vous pouvez créer la base via l'interface phpMyAdmin ou exécuter le script SQL ci-dessous.

Table Contacts :

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int			No	None		AUTO_INCREMENT	Change Drop More
☐	2 nom	varchar(100)	utf8mb4_unicode_ci		No	None			Change Drop More
☐	3 email	varchar(150)	utf8mb4_unicode_ci		No	None			Change Drop More
☐	4 telephone	varchar(30)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
☐	5 ville	varchar(80)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
☐	6 created_at	timestamp			Yes	CURRENT_TIMESTAMP		DEFAULT_GENERATED	Change Drop More

Table messages :

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int			No	None		AUTO_INCREMENT	Change Drop More
☐	2 contact_id	int			No	None			Change Drop More
☐	3 contenu	text	utf8mb4_unicode_ci		No	None			Change Drop More
☐	4 created_at	timestamp			Yes	CURRENT_TIMESTAMP		DEFAULT_GENERATED	Change Drop More

Ajouter 2 lignes dans la table contacts

		id	nom	email	telephone	ville	created_at
☐	Edit Copy Delete	1	Alice Dupont	alice@d-clic.com	22112233	Mali	2026-03-15 17:24:11
☐	Edit Copy Delete	2	Bob Martin	bob@d-clic.com	55119988	Niger	2026-03-15 17:24:11

☐ Check all
 With selected: Edit Copy Delete Export

Solution A3 — Script SQL (à exécuter dans phpMyAdmin)

```
-- Base : gestion_contacts
CREATE DATABASE IF NOT EXISTS gestion_contacts
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE gestion_contacts;
```



```

CREATE TABLE IF NOT EXISTS contacts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  telephone VARCHAR(30) NULL,
  ville VARCHAR(80) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  contact_id INT NOT NULL,
  contenu TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_messages_contact
    FOREIGN KEY (contact_id) REFERENCES contacts(id)
    ON DELETE CASCADE
);

-- Données de test
INSERT INTO contacts (nom, email, telephone, ville) VALUES
('Alice Dupont', 'alice@example.com', '22112233', 'Tunis'),
('Bob Martin', 'bob@example.com', '55119988', 'Sfax');

INSERT INTO messages (contact_id, contenu) VALUES
(1, 'Bonjour, je souhaite plus d'infos.'),
(2, 'Merci pour votre retour.');
```

2.6 Vérification (phpMyAdmin)

- Dans phpMyAdmin : vérifier que la base gestion_contacts existe.
- Ouvrir la table contacts : vérifier 2 lignes insérées.
- Ouvrir la table messages : vérifier 2 lignes insérées.
- Exécuter une requête SELECT dans l'onglet SQL pour tester : `SELECT * FROM contacts;`

3. Volet B — Activité principale : SQL + Connexion PHP (PDO & MySQLi)

3.1. Contexte (énoncé)

Vous intervenez comme développeur back-end sur une mini-application de gestion de formations. Votre mission est de concevoir la base de données relationnelle à partir d'un besoin métier, puis de la créer dans MySQL et enfin de tester une connexion depuis PHP.

3.2. Description du besoin

Une structure de formation souhaite gérer : les apprenants, les formations, les sessions de formation, et les inscriptions. Les règles de gestion suivantes doivent être respectées :

- Un apprenant est défini au minimum par : nom, prénom, email, téléphone (optionnel), date de naissance (optionnel), date d'inscription.
- Une formation est définie par : titre, description courte, niveau (débutant/intermédiaire/avancé), durée (heures).
- Une formation peut être proposée en plusieurs sessions (ex. session avril, session juin).
- Une session est définie par : date de début, date de fin, lieu (présentiel ou en ligne), nombre de places.
- Un apprenant peut s'inscrire à plusieurs sessions ; une session peut accueillir plusieurs apprenants.
- Une inscription contient : date d'inscription, statut (en_attente/confirmée/annulée), et éventuellement une note finale (0–20) si l'apprenant a terminé.
- Contrainte : un apprenant ne peut pas être inscrit deux fois à la même session.
- Contrainte : l'email d'un apprenant doit être unique.
- Contrainte : le nombre d'inscriptions confirmées ne doit pas dépasser le nombre de places (à traiter au moins par logique applicative ou contrainte si vous savez le faire).

3.3. Travail demandé

1. Identifier les entités et leurs attributs (liste claire).
2. Identifier les associations entre entités et préciser les cardinalités (1..N, N..N, etc.).
3. Proposer un schéma logique (tables + clés primaires + clés étrangères).

4. Donner le schéma physique : types MySQL (INT, VARCHAR, DATE, DATETIME...), contraintes (NOT NULL, UNIQUE...), index si pertinent.
5. Créer la base dans phpMyAdmin (ou via un script SQL).
6. Insérer des données de test (au moins 5 apprenants, 3 formations, 3 sessions, 8 inscriptions).
7. Écrire un script PHP de connexion à la base en utilisant PDO OU MySQLi.

3.4. Livrables attendus (dépôt)

- Un fichier .zip nommé : NOM_Prenom_S3_Donnees_ActiviteB.zip
- Un dossier /conception/ : MCD (image ou PDF), MLD (texte ou schéma), MPD (schéma tables).
- Un dossier /sql/ : schema.sql (CREATE DATABASE + CREATE TABLE) contenant l'export de votre base de données à partir de PhpMyAdmin.
- Un dossier /php/ : connect_pdo.php (ou connect_mysqli.php).
- Un README.txt : comment importer la base, quels identifiants utiliser, et quelles URL tester.